

Drone AutoFly

```
from dronekit import connect, VehicleMode, LocationGlobalRelative, Command
```

```
from pymavlink import mavutil
```

```
import time
```

```
# 1. CONNECT TO THE DRONE
```

```
# Change '127.0.0.1:14550' to '/dev/ttyAMA0' if using a Raspberry Pi
```

```
vehicle = connect('127.0.0.1:14550', wait_ready=True)
```

```
def arm_and_takeoff(target_altitude):
```

```
    print("Pre-arm checks...")
```

```
    while not vehicle.is_armable:
```

```
        time.sleep(1)
```

```
    print("Arming motors...")
```

```
    vehicle.mode = VehicleMode("GUIDED")
```

```
    vehicle.armed = True
```

```
    while not vehicle.armed:
```

```
        time.sleep(1)
```

```
    print(f"Taking off to {target_altitude}m")
```

```
    vehicle.simple_takeoff(target_altitude)
```

```
    while True:
```

```
        if vehicle.location.relative_alt >= target_altitude * 0.95:
```

```
            print("Reached altitude")
```

```
            break
```

```
            time.sleep(1)
```

```
# 2. MOVE TO WAYPOINT
```

```

def goto_waypoint(lat, lon, alt):
    print(f'Moving to Lat: {lat}, Lon: {lon}')
    point = LocationGlobalRelative(lat, lon, alt)
    vehicle.simple_goto(point)
    # Logic to wait until drone reaches the point
    time.sleep(10)

# 3. SET CAMERA GIMBAL ANGLE
def set_gimbal_angle(pitch):
    print(f'Setting gimbal pitch to {pitch} degrees')
    # MAV_CMD_DO_MOUNT_CONTROL: pitch, roll, yaw
    msg = vehicle.message_factory.command_long_encode(
        0, 0, # target system, target component
        mavutil.mavlink.MAV_CMD_DO_MOUNT_CONTROL,
        0, # confirmation
        pitch, # param1: pitch
        0, # param2: roll
        0, # param3: yaw
        0, 0, 0, # param 4-6
        mavutil.mavlink.MAV_MOUNT_MODE_MAVLINK_TARGETING)
    vehicle.send_mavlink(msg)

# --- THE SURVEILLANCE MISSION ---

# Step 1: Takeoff to 30m
arm_and_takeoff(30)

# Step 2: Move to first waypoint at 50m
goto_waypoint(45.123, -93.456, 50)

# Step 3: Tilt camera down 45 degrees

```

```

set_gimbal_angle(-45)

# Step 4: Loiter/Scan for 30 seconds
print("Scanning forest area...")
time.sleep(30)

# Step 5: Move to second waypoint
goto_waypoint(45.125, -93.458, 50)

# Step 6: Return to Launch
print("Mission complete. Returning to Home.")
vehicle.mode = VehicleMode("RTL")

# Close connection
vehicle.close()

```

For GPS fix and Takeoff

```

from dronekit import connect, VehicleMode, LocationGlobalRelative
import time

# 1. Connect to the Vehicle (UDP for Simulation or Serial for Raspberry Pi)
# For Pi use: '/dev/ttyAMA0', baud=57600
connection_string = '127.0.0.1:14550'
print(f"Connecting to vehicle on: {connection_string}")
vehicle = connect(connection_string, wait_ready=True)

def arm_and_takeoff(target_altitude):
    print("Basic pre-arm checks...")
    # Don't try to arm until autopilot is ready

```

```

while not vehicle.is_armable:
    print(" Waiting for vehicle to initialise...")
    time.sleep(1)

print("Arming motors")
vehicle.mode = VehicleMode("GUIDED")
vehicle.armed = True

while not vehicle.armed:
    print(" Waiting for arming...")
    time.sleep(1)

print(f"Taking off! Target Altitude: {target_altitude}")
vehicle.simple_takeoff(target_altitude)

# Wait until the vehicle reaches a safe height
while True:
    print(f" Current Altitude: {vehicle.location.relative_alt}")
    if vehicle.location.relative_alt >= target_altitude * 0.95:
        print("Reached target altitude")
        break
    time.sleep(1)

# --- EXECUTION ---
arm_and_takeoff(20) # Take off to 20 meters for forest clearing

print("Setting groundspeed to 5m/s...")
vehicle.groundspeed = 5

```

```
# Disconnect after mission
# vehicle.close()
```

For Showing Battery

```
from dronekit import connect
import time

# Connect to the Flight Controller
vehicle = connect('127.0.0.1:14550', wait_ready=True)

def monitor_battery():
    while True:
        # Fetch battery attributes
        voltage = vehicle.battery.voltage
        current = vehicle.battery.current
        level = vehicle.battery.level # Percentage remaining

        print(f"--- Battery Status ---")
        print(f"Voltage: {voltage}V")
        print(f"Current Draw: {current}A")
        print(f"Percentage: {level}%")

        # Basic Logic for Forest Safety
        if level < 25:
            print("WARNING: Battery low! Consider returning to launch.")

        if level < 15:
            print("CRITICAL: Battery critical! Emergency landing or RTL required.")
            # vehicle.mode = VehicleMode("RTL") # Uncomment to automate return
```

```
time.sleep(2) # Update every 2 seconds

# Start monitoring

try:
    monitor_battery()
except KeyboardInterrupt:
    print("Monitoring stopped.")
    vehicle.close()
```

To Connect With ML or DL

```
import cv2

from ultralytics import YOLO

from dronekit import connect, VehicleMode

import time


# 1. Connect to the Drone and Load the AI Model

vehicle = connect('/dev/ttyAMA0', baud=57600) # Physical connection
model = YOLO('forest_surveillance_model.pt') # Your trained DL model


# 2. Open Camera Stream

cap = cv2.VideoCapture(0)

print("AI Surveillance System Active...")

while cap.isOpened():
    success, frame = cap.read()

    if not success:
        break
```

3. RUN MACHINE LEARNING INFERENCE

```
results = model(frame, conf=0.5) # Detect objects with >50% confidence
```

```
for r in results:
```

```
    for box in r.bboxes:
```

```
        # Get Label (e.g., 0 = 'Smoke', 1 = 'Human')
```

```
        class_id = int(box.cls[0])
```

```
        label = model.names[class_id]
```

```
        print(f"ALERT: {label} detected!")
```

4. ACTION BASED ON DETECTION

```
if label == "smoke":
```

```
    print("Fire detected! Switching to HOVER mode for inspection.")
```

```
    vehicle.mode = VehicleMode("LOITER")
```

```
    # Optional: Send GPS coordinates to ground station
```

```
    print(f"Location: {vehicle.location.global_frame}")
```

```
# (Optional) Display the video feed if a monitor is connected
```

```
# cv2.imshow("Drone AI Feed", results[0].plot())
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

```
cap.release()
```

```
vehicle.close()
```

Real Autonomy

```
from dronekit import connect, VehicleMode
```

```
import time
```

```

class ForestDrone:

    def __init__(self):

        self.vehicle = connect('127.0.0.1:14550', wait_ready=True)

        self.state = "SEARCH" # Initial State


    def run(self):

        while True:

            if self.state == "SEARCH":

                self.handle_search()

            elif self.state == "TRACK":

                self.handle_track()

            elif self.state == "INVESTIGATE":

                self.handle_investigate()

            elif self.state == "RETURN_HOME":

                self.handle_return()

            elif self.state == "EMERGENCY_LAND":

                self.handle_emergency()


            # Global Failsafe: Check battery in every loop

            if self.vehicle.battery.level < 15:

                self.state = "EMERGENCY_LAND"


            time.sleep(1)


    def handle_search(self):

        print("State: SEARCHING - Flying grid pattern...")

        # Code to move to next waypoint

        # If ML model detects something:

        # self.state = "INVESTIGATE"

        pass

```



```

def handle_investigate(self):
    print("State: INVESTIGATE - Dropping altitude to 15m for closer look...")
    self.vehicle.simple_goto(target_location, groundspeed=2)
    # If confirmed: self.state = "TRACK"
    # If false alarm: self.state = "SEARCH"
    pass

def handle_track(self):
    print("State: TRACKING - Following moving target...")
    # Use ML bounding box to adjust drone position
    pass

def handle_return(self):
    print("State: RETURNING - Mission complete.")
    self.vehicle.mode = VehicleMode("RTL")

def handle_emergency(self):
    print("State: EMERGENCY - Landing immediately!")
    self.vehicle.mode = VehicleMode("LAND")

# Start the drone
my_drone = ForestDrone()
my_drone.run()

```

Boustrophedon (Lawn-mower) Grid Generation

```

import numpy as np
from shapely.geometry import Polygon, LineString

def generate_grid(polygon_coords, spacing=20):
    """
    polygon_coords: List of (lat, lon) tuples defining the forest boundary
    """

```

spacing: Distance between passes in meters (approx 0.0002 degrees)

"""

poly = Polygon(polygon_coords)

minx, miny, maxx, maxy = poly.bounds

waypoints = []

Create parallel lines across the bounding box

for i, x in enumerate(np.arange(minx, maxx, spacing * 0.00001)):

line = LineString([(x, miny), (x, maxy)])

intersection = line.intersection(poly)

if not intersection.is_empty:

Extract points and flip order every other line for "snake" pattern

coords = list(intersection.coords)

if i % 2 == 1:

coords.reverse()

waypoints.extend(coords)

return waypoints # List of (lat, lon) for the SEARCH state

Adaptive Altitude (Terrain Following)

def adaptive_altitude_check(target_agl=30):

"""

Adjusts altitude based on real-time Lidar/Rangefinder readings.

"""

Get distance to the canopy/ground from the Lidar sensor

current_agl = vehicle.rangefinder.distance

if current_agl is None: return # Sensor error

error = target_agl - current_agl

```

# If we are too low (e.g., a hill or tall tree), climb immediately
if error > 2: # 2 meter tolerance
    new_alt = vehicle.location.relative_alt + error
    print(f"Terrain rising! Adjusting altitude to {new_alt}m")

# Command the drone to climb while maintaining horizontal mission
point = LocationGlobalRelative(
    vehicle.location.global_frame.lat,
    vehicle.location.global_frame.lon,
    new_alt
)
vehicle.simple_goto(point)

```

Integrated State Machine Logic

```

def handle_search(self):
    print("State: SEARCHING - Executing Grid Pattern")

    # 1. Generate the grid if not already done
    if not self.mission_waypoints:
        self.mission_waypoints = generate_grid(self.boundary_polygon)

    # 2. Navigate to the next point in the lawn-mower pattern
    target = self.mission_waypoints[self.current_idx]
    self.vehicle.simple_goto(LocationGlobalRelative(target[0], target[1], self.base_alt))

    # 3. Terrain Following: Keep altitude safe above trees
    adaptive_altitude_check(target_agl=30)

    # 4. Check for target detection (ML Bridge)
    if self.object_detected:

```

```
self.state = "INVESTIGATE"
```

Multi-frame Confirmation (N-out-of-M Rule)

```
from collections import deque
```

```
class DetectionFilter:
```

```
    def __init__(self, n=8, m=10):
```

```
        self.n = n
```

```
        self.m = m
```

```
        self.buffer = deque(maxlen=m)
```

```
    def update(self, detected_bool):
```

```
        self.buffer.append(detected_bool)
```

```
        # Check if true detections meet the threshold
```

```
        if self.buffer.count(True) >= self.n:
```

```
            return True
```

```
        return False
```

```
# Usage in your loop:
```

```
# smoke_filter = DetectionFilter(n=8, m=10)
```

```
# if smoke_filter.update(ml_result_is_smoke):
```

```
#     drone.state = "INVESTIGATE"
```

Timestamped Detections (Logging)

```
import datetime
```

```
import csv
```

```
def log_detection(label, vehicle):
```

```
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```

lat = vehicle.location.global_frame.lat
lon = vehicle.location.global_frame.lon
alt = vehicle.location.global_frame.alt

with open('forest_log.csv', mode='a') as file:
    writer = csv.writer(file)
    writer.writerow([timestamp, label, lat, lon, alt])
print(f"Logged {label} at {lat}, {lon}")

```

Heatmap & Threat Clustering

```

from sklearn.cluster import DBSCAN
import numpy as np

# Example: List of [lat, lon] detections from the flight
detection_points = np.array([
    [45.1231, -93.4561],
    [45.1232, -93.4562],
    [45.1250, -93.4580], # Outlier
])

def get_clusters(points):
    # eps is the max distance between two samples (approx meters in degrees)
    clustering = DBSCAN(eps=0.0001, min_samples=3).fit(points)

    unique_labels = set(clustering.labels_)
    clusters = []

    for label in unique_labels:
        if label == -1: continue # Noise
        # Calculate center of the cluster
        class_member_mask = (clustering.labels_ == label)

```

```

cluster_center = points[class_member_mask].mean(axis=0)
clusters.append(cluster_center)

return clusters # Returns a list of central "threat" coordinates

```

Automated Failsafe Logic

```

def safety_supervisor(vehicle):

    # 1. LINK-LOSS HANDLING (GCS Disconnect)

    # Most FCs handle this at the hardware level, but we can monitor it in Python
    if not vehicle.last_heartbeat < 5: # If no heartbeat for 5 seconds
        print("CRITICAL: Link Loss Detected!")

        # The hardware usually triggers RTL automatically if configured

    # 2. GPS LOSS HANDLING

    # If GPS glitch occurs, the drone cannot RTL (it doesn't know where it is).
    # We must switch to 'LAND' or 'ALT_HOLD' to prevent drifting.
    if vehicle.gps_0.fix_type < 2: # Lower than 3D Fix
        print("ALARM: GPS Loss! Switching to Emergency LAND.")
        vehicle.mode = VehicleMode("LAND")

    # 3. BATTERY FAILSAFE

    if vehicle.battery.level < 15:
        print("Low Battery: Initiating Automatic RTL.")
        vehicle.mode = VehicleMode("RTL")

```

GPS-Denied

```

def check_gps_quality(vehicle):

    # HDOP (Horizontal Dilution of Precision) - lower is better
    if vehicle.gps_0.eph > 200: # eph is in cm; 200 = 2 meters
        print("GPS Accuracy degrading. Pausing mission to hover.")
        vehicle.mode = VehicleMode("LOITER")

```

Emergency LAND vs. RTL Logic

```
def trigger_emergency(reason):  
    print(f"EMERGENCY: {reason}")  
  
    # Set the 'Failsafe' bit in the flight controller  
    vehicle.mode = VehicleMode("LAND")  
  
    # Log coordinates for recovery  
    log_detection("EMERGENCY_LAND_LOCATION", vehicle)
```

The Server: FastAPI Endpoint

```
from fastapi import FastAPI, UploadFile, File, Form  
  
import shutil  
import os  
import json  
  
app = FastAPI()  
  
# Directory to store snapshots of forest threats  
UPLOAD_DIR = "alerts"  
os.makedirs(UPLOAD_DIR, exist_ok=True)  
  
@app.post("/report_threat")  
async def report_threat(  
    packet: str = Form(...),    # JSON string  
    image: UploadFile = File(...) # The JPG file  
):  
    # 1. Parse JSON data  
    data = json.loads(packet)  
    threat_type = data['label']  
  
    # 2. Save the Image
```

```

file_path = f"{UPLOAD_DIR}/{data['timestamp']}_{threat_type}.jpg"
with open(file_path, "wb") as buffer:
    shutil.copyfileobj(image.file, buffer)

print(f"ALERT RECEIVED: {threat_type} at {data['lat']}, {data['lon']}")
return {"status": "received", "image_path": file_path}

```

The Client: Drone Side (JSON + Image Upload)

```

import requests
import json
import cv2

def upload_alert(vehicle, label, frame):
    url = "http://base-station-ip:8000/report_threat"

    # 1. Create JSON Alert Packet
    payload = {
        "timestamp": datetime.datetime.now().strftime("%H%M%S"),
        "label": label,
        "lat": vehicle.location.global_frame.lat,
        "lon": vehicle.location.global_frame.lon,
        "alt": vehicle.location.global_frame.alt,
        "battery": vehicle.battery.level
    }

    # 2. Prepare Image Snapshot
    # Encode frame to JPG in memory
    _img_encoded = cv2.imencode('.jpg', frame)

    files = {

```



```
"image": ("snapshot.jpg", img_encoded.tobytes(), "image/jpeg")
}
```

3. Send REST API Call

try:

```
response = requests.post(url, data={"packet": json.dumps(payload)}, files=files, timeout=5)
```

```
return response.status_code == 200
```

except Exception as e:

```
print(f"Failed to send alert: {e}")
```

```
return False
```

Thermal Thresholding (OpenCV)

```
import cv2
```

```
import numpy as np
```

```
def detect_heat_sources(thermal_frame, temp_threshold=100):
```

```
    """
```

```
    thermal_frame: Grayscale array where pixel value = temperature
```

```
    temp_threshold: Degrees Celsius to trigger an alert
```

```
    """
```

```
# 1. Apply threshold: pixels above temp_threshold become white (255)
```

```
_ , thermal_binary = cv2.threshold(thermal_frame, temp_threshold, 255, cv2.THRESH_BINARY)
```

```
# 2. Clean up noise (remove small hot pixels)
```

```
kernel = np.ones((5,5), np.uint8)
```

```
thermal_binary = cv2.morphologyEx(thermal_binary, cv2.MORPH_OPEN, kernel)
```

```
# 3. Find contours (blobs of heat)
```

```
contours, _ = cv2.findContours(thermal_binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

```
heat_blobs = []
```

```
for cnt in contours:
```

```

x, y, w, h = cv2.boundingRect(cnt)

if w * h > 50: # Minimum size to ignore birds/small animals
    heat_blobs.append((x, y, w, h))

return heat_blobs, thermal_binary

```

Sensor Fusion Logic

```

def sensor_fusion_decision(rgb_detections, thermal_detections):
    """
    Combines visual labels (YOLO) with thermal blobs.
    """
    is_confirmed_threat = False
    priority = "LOW"

    # Example Logic
    has_smoke_visual = any(d['label'] == 'smoke' for d in rgb_detections)
    has_heat_blob = len(thermal_detections) > 0

    if has_smoke_visual and has_heat_blob:
        priority = "CRITICAL" # Likely an active fire
        is_confirmed_threat = True
    elif has_heat_blob:
        priority = "MEDIUM" # Possible trespasser or campfire
        is_confirmed_threat = True
    elif has_smoke_visual:
        priority = "LOW" # Likely fog or low-risk anomaly

    return is_confirmed_threat, priority

```

Handling Separate Feeds (Multithreading)

```
import threading

class DualCameraStream:

    def __init__(self):
        self.rgb_frame = None
        self.thermal_frame = None
        self.running = True

    def start_rgb(self):
        cap = cv2.VideoCapture(0) # Visual Camera
        while self.running:
            _, self.rgb_frame = cap.read()

    def start_thermal(self):
        cap = cv2.VideoCapture(1) # Thermal Camera
        while self.running:
            _, self.thermal_frame = cap.read()

# Start both streams in the background
streams = DualCameraStream()
threading.Thread(target=streams.start_rgb).start()
threading.Thread(target=streams.start_thermal).start()
```

Object Tracking: DeepSORT Integration

```
from deep_sort_realtime.deepsort_tracker import DeepSort

tracker = DeepSort(max_age=30) # Keep track of ID for 30 frames after loss
```

```

def process_tracking(frame, detections):
    # detections: list of [[x, y, w, h], confidence, class_id]
    tracks = tracker.update_tracks(detections, frame=frame)

    for track in tracks:
        if not track.is_confirmed():
            continue

        track_id = track.track_id

        ltrb = track.to_ltrb() # Get bounding box

        # Log movement vector
        print(f"Tracking ID: {track_id} at {ltrb}")

```

Edge Optimization: TensorRT & Frame Skipping

```

import time

prev_time = 0
frame_count = 0

while True:
    ret, frame = cap.read()

    frame_count += 1

    # FRAME SKIPPING: Only run AI on every 3rd frame
    if frame_count % 3 != 0:
        continue

    # FPS MONITORING
    curr_time = time.time()
    fps = 1 / (curr_time - prev_time)
    prev_time = curr_time

```

```
# TENSORRT LOGGING (Conceptual)

# Using a .engine file instead of .pt reduces latency by ~4x

results = tensorrt_model.inference(frame)
```

Multi-Drone Readiness (Swarm Logic)

```
{
  "drone_id": "FOREST_UNIT_02",
  "status": "TRACKING",
  "zone": "SE_QUADRANT",
  "proximity_alert": ["FOREST_UNIT_01"]
}
```

Collision-Free Zones (Geofencing)

```
def check_proximity(my_loc, other_drone_loc):
    distance = calculate_distance(my_loc, other_drone_loc)

    if distance < 50: # 50-meter safety bubble
        print("COOPERATIVE AVOIDANCE: Adjusting Altitude")
        vehicle.simple_goto(LocationGlobalRelative(lat, lon, alt + 10))
```